Job Mansion: Ed Tech Platform Project Guide

MERN stack | Two roles (User and Admin) | Razorpay payments | Built on free tiers, ready to scale

1. Project Overview

Job Mansion is an Ed Tech website that offers training in different software courses. Visitors can browse the course catalog, create an account, pay for a course online, and access their purchased courses. An admin manages courses, enquiries, students and payments from a protected dashboard.

The structure is inspired by common course-platform layouts (course catalog, course detail pages, enrollment flow). All text, images and branding must be original or supplied by the client.

In scope

- Public website: Home, Courses, Course Detail, About, Contact
- Authentication with exactly two roles: **User** (student) and **Admin**
- Online payment for courses (Razorpay)
- User dashboard: My Courses, payment history, profile
- Admin dashboard: manage courses, students, enrollments, payments, enquiries
- Contact and demo-request form

Out of scope (for now)

- Trainer or instructor accounts, multi-admin permission levels
- Live classes, quizzes, certificates, discussion forums
- Mobile app
- Self-hosted video streaming (use unlisted YouTube or Vimeo links)

2. Roles and Permissions

Capability	Guest	User	Admin
Browse courses, view details	Yes	Yes	Yes
Submit enquiry or demo request	Yes	Yes	Yes

Capability	Guest	User	Admin
Register, log in	Yes	n/a	n/a
Buy a course	No	Yes	No
View purchased courses and lessons	No	Yes (own only)	Yes (all)
View own payment history	No	Yes	Yes (all)
Edit own profile, change password	No	Yes	Yes
Create, edit, delete courses and lessons	No	No	Yes
View and manage students	No	No	Yes
View all payments, issue refunds	No	No	Yes
View and update enquiries	No	No	Yes

The role is stored as a single field on the user record (`role: 'user' | 'admin'`). The first admin is created by a seed script, never through public registration.

3. High-Level Architecture

```
Browser (React SPA)
       |
       |  HTTPS (JSON, JWT)
       v
Express API (Node.js)  ------>  Razorpay (orders, checkout, webhooks)
  |        |        |
  |        |        +-------->  Email service (Nodemailer / Resend)
  |        +----------------->  Cloudinary (images)
  v
MongoDB Atlas
```

Layers

- **Client:** React single-page app. Talks to the API only through one Axios instance. Holds the short-lived access token in memory.
- **API:** Express app organized as routes, then controllers, then services, then models. Middleware handles authentication, role checks, validation, rate limiting and errors.
- **Database:** MongoDB with Mongoose models and indexes.
- **Third parties:** Razorpay for payments, Cloudinary for images, an email provider for receipts and enquiry alerts.

Request flow example (viewing a purchased course)

1. React sends `GET /api/courses/:id/content` with the access token.
2. `authenticate` middleware verifies the JWT and loads the user.
3. The controller checks that an `Enrollment` exists for this user and course (admins skip this check).
4. The service returns lessons; the client renders them.

4. Technology Stack

Area	Choice	Notes
Frontend	React 18 with Vite	Fast builds, easy deploys
Routing	React Router	Protected and admin-only routes
Styling	Tailwind CSS	Responsive, consistent UI
State and data	React Query plus Context	Server cache plus auth state
Forms	React Hook Form with Zod	Validation on the client
Backend	Node.js with Express	REST API
Database	MongoDB Atlas (M0 free) with Mongoose	512 MB is plenty for Phase 1
Auth	JWT, bcrypt, httpOnly cookies	See section 7
Validation	Zod or Joi	Same rules on every endpoint
Payments	Razorpay	Standard for India: UPI, cards, netbanking
Images	Cloudinary free tier	Course thumbnails and banners
Email	Nodemailer (Gmail SMTP) or Resend free tier	Receipts, enquiry alerts
Hosting	Vercel or Netlify (frontend), Render (backend)	Free tiers
Domain	GoDaddy	DNS only; point records to the hosts above
Source control	GitHub	Private repo until payment is settled

5. Folder Structure

Backend

```
server/
  src/
    config/        db.js, env.js, razorpay.js, cloudinary.js
    models/        User.js, Course.js, Enrollment.js, Payment.js, Enquiry.js
    routes/        auth.routes.js, course.routes.js, payment.routes.js,
                   user.routes.js, admin.routes.js, enquiry.routes.js
    controllers/   one file per route group
    services/      payment.service.js, email.service.js,
enrollment.service.js
    middleware/    authenticate.js, authorize.js, validate.js,
                   rateLimit.js, errorHandler.js
    validators/    zod schemas per resource
    utils/         generateToken.js, apiError.js, asyncHandler.js
    seed/          createAdmin.js, sampleCourses.js
    app.js         express setup, middleware, routes
    server.js      starts the server
  .env.example
  package.json
```

Frontend

```
client/
  src/
    api/           axios instance, one file per resource
    components/    ui/, layout/, course/, forms/
    context/       AuthContext.jsx
    hooks/         useAuth, useCourses, usePayment
    pages/
      public/      Home, Courses, CourseDetail, About, Contact, Login,
Register
      user/        Dashboard, MyCourses, CoursePlayer, Payments, Profile
      admin/       Dashboard, Courses, CourseForm, Students, Payments,
Enquiries
    routes/        ProtectedRoute.jsx, AdminRoute.jsx, index.jsx
    utils/         formatCurrency, formatDate
    main.jsx
  .env.example
  package.json
```

6. Database Design

Five collections cover the whole product.

User

Field	Type	Notes
name	String	required
email	String	required, unique, lowercase
phone	String	optional
passwordHash	String	bcrypt, never returned by the API
role	String	'user' or 'admin', default 'user'
isActive	Boolean	admin can block an account
refreshTokenHash	String	for refresh-token rotation
createdAt, updatedAt	Date	timestamps

Course

Field	Type	Notes
title	String	required
slug	String	unique, used in URLs
category	String	for example Full Stack, Data, Cloud, Testing
shortDescription, description	String	
thumbnailUrl	String	Cloudinary URL
price	Number	stored in paise (integer) to avoid rounding errors
discountPrice	Number	optional
duration	String	for example '3 months'
mode	String	online, offline or hybrid
level	String	beginner, intermediate, advanced
syllabus	Array	modules with a title and topics

Field	Type	Notes
lessons	Array	`{ title, videoUrl, order, isPreview }`
isPublished	Boolean	drafts stay hidden
createdAt, updatedAt	Date	

Indexes: `slug` (unique), `category`, `isPublished`, and a text index on `title` and `description` for search.

Payment

Field	Type	Notes
user	ObjectId	ref User
course	ObjectId	ref Course
amount	Number	in paise
currency	String	'INR'
razorpayOrderId	String	unique
razorpayPaymentId	String	set after payment
razorpaySignature	String	stored for audit
status	String	created, paid, failed, refunded
method	String	upi, card, netbanking (from Razorpay)
createdAt, updatedAt	Date	

Enrollment

Field	Type	Notes
user	ObjectId	ref User
course	ObjectId	ref Course
payment	ObjectId	ref Payment
status	String	active or revoked
enrolledAt	Date	

A unique compound index on `(user, course)` prevents duplicate enrollments.

Enquiry

Field	Type	Notes
name, email, phone	String	
course	ObjectId	optional, which course they asked about
message	String	
type	String	'enquiry' or 'demo'
status	String	new, contacted, closed
createdAt	Date	

7. Authentication and Authorization

Approach

- Registration and login with email and password. Passwords hashed with bcrypt (cost factor 10 to 12).
- On login the server issues a **short-lived access token** (about 15 minutes) and a **refresh token** (about 7 days).
- The refresh token is sent as an **httpOnly, secure, sameSite cookie** so JavaScript cannot read it. The access token is kept in memory on the client.
- When the access token expires, the client calls `POST /api/auth/refresh`; the server rotates the refresh token and stores only its hash.
- Logout clears the cookie and the stored hash.

Middleware

- `authenticate`: reads the bearer token, verifies it, attaches `req.user`. Rejects blocked accounts.
- `authorize('admin')`: allows the route only if `req.user.role` matches.
- Every admin route is mounted behind both: `router.use(authenticate, authorize('admin'))`.

Frontend guards

- `ProtectedRoute` redirects guests to login and returns them to the page they wanted.

- `AdminRoute` additionally checks `role === 'admin'`.
- Frontend guards are for user experience only. The real enforcement is always on the server.

Password reset

User requests a reset, server emails a single-use token link (valid for 15 to 30 minutes, stored hashed), user sets a new password and all refresh tokens are cleared.

8. REST API Design

Base path: `/api`. Responses are JSON with a consistent shape: `{ success, data, message }`.

Auth

Method	Endpoint	Access	Purpose
POST	/auth/register	Public	Create a user account
POST	/auth/login	Public	Log in, receive tokens
POST	/auth/refresh	Cookie	Issue a new access token
POST	/auth/logout	User	Clear tokens
POST	/auth/forgot-password	Public	Send reset email
POST	/auth/reset-password	Public	Set a new password

Courses (public)

Method	Endpoint	Purpose
GET	/courses	List published courses; supports `?category=&search=&page=&limit=`
GET	/courses/:slug	Course details with syllabus (lessons hidden unless enrolled)
GET	/courses/:id/content	Lessons for enrolled users and admins

User

Method	Endpoint	Purpose
GET	/users/me	Own profile
PUT	/users/me	Update profile
PUT	/users/me/password	Change password
GET	/users/me/enrollments	My courses
GET	/users/me/payments	My payment history

Payments

Method	Endpoint	Access	Purpose
POST	/payments/create-order	User	Create a Razorpay order for a course
POST	/payments/verify	User	Verify signature and activate enrollment
POST	/payments/webhook	Razorpay	Server-to-server confirmation (raw body)

Enquiries

Method	Endpoint	Access	Purpose
POST	/enquiries	Public (rate limited)	Submit enquiry or demo request

Admin (all require the admin role)

Method	Endpoint	Purpose
GET	/admin/stats	Totals: students, revenue, enrollments, new enquiries
POST, PUT, DELETE	/admin/courses	Create, update, delete (or unpublish) courses
POST	/admin/upload	Upload a thumbnail to Cloudinary
GET	/admin/users	List students, search, paginate
PATCH	/admin/users/:id/status	Block or unblock a user

Method	Endpoint	Purpose
GET	/admin/payments	All payments with filters
POST	/admin/payments/:id/refund	Refund through Razorpay
POST	/admin/enrollments	Manually enroll a user (offline or cash payments)
PATCH	/admin/enrollments/:id	Revoke or restore access
GET, PATCH	/admin/enquiries	List and update status

9. Payment Integration (Razorpay)

Razorpay supports UPI, cards and netbanking and has a free **test mode**, so the whole flow can be built and demoed without paying anything. Going live needs the client's business KYC (PAN, bank account and business details), and Razorpay charges a per-transaction fee; confirm current pricing on their site.

Golden rules

1. **The price always comes from the database, never from the browser.**
2. **The key secret never leaves the server.** Only the public key id goes to the frontend.
3. **Never trust the checkout success callback alone.** Verify the signature on the server, and also handle the webhook.
4. **Make every step idempotent** so a repeated callback or webhook cannot create duplicate enrollments.

Flow

```
User clicks Buy
  |
  v
POST /payments/create-order   (server reads price from DB, creates
  |                            Razorpay order, saves Payment: created)
  v
Razorpay Checkout opens in the browser (key id + order id)
  |
  v
User pays  -->  Razorpay returns payment id + signature to the browser
  |
  v
POST /payments/verify          (server checks HMAC signature, marks
  |                            Payment: paid, creates Enrollment)
  v
Webhook (payment.captured / order.paid) arrives later as a backup
```

Step 1: Create the order (server)

- Check the course exists, is published, and the user is not already enrolled.
- Create the order with the amount in paise, currency INR, and a `receipt` such as the Payment document id.
- Save a `Payment` record with status `created` and the `razorpayOrderId`.
- Return `{ orderId, amount, currency, keyId }` to the client.

Step 2: Open checkout (client)

- Load the Razorpay checkout script, then open it with the key id, order id, the course name, and the user's name, email and phone as prefill.
- In the success handler, send `razorpay_order_id`, `razorpay_payment_id` and `razorpay_signature` to `/payments/verify`.

Step 3: Verify (server)

The expected signature is an HMAC SHA256 of `orderId + '|' + paymentId` using the key secret:

```
const crypto = require('crypto');
const expected = crypto
  .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET)
  .update(orderId + '|' + paymentId)
  .digest('hex');
const valid = crypto.timingSafeEqual(
  Buffer.from(expected), Buffer.from(signature)
);
```

If valid and the payment is not already `paid`, update the payment (status, payment id, signature), create the `Enrollment`, and send a receipt email. If invalid, mark the payment `failed` and return an error.

Step 4: Webhook (backup)

- Configure a webhook in the Razorpay dashboard pointing to `/api/payments/webhook` with a webhook secret.
- Use `express.raw()` on this route so the signature can be checked against the exact raw body.
- Handle `payment.captured` or `order.paid` by running the same idempotent activation logic. This covers users who pay and then close the browser before the callback.
- Handle `payment.failed` and refund events to keep statuses accurate.

Refunds and records

- Admin triggers a refund from the dashboard; the server calls Razorpay's refund API, sets the payment to `refunded` and the enrollment to `revoked`.
- Keep all payment records permanently for accounting. Consider generating a simple invoice PDF later (GST details come from the client).

Testing checklist

- Successful payment in test mode (use Razorpay's test cards and test UPI ids)
- Failed or cancelled payment leaves no enrollment
- Replaying the same verify request does not duplicate the enrollment
- Tampered signature is rejected
- Webhook alone (without the browser callback) activates the enrollment
- Already-enrolled user cannot buy the same course twice

10. Frontend Pages and Routes

Public

Route	Page	Content
/	Home	Hero, popular courses, why choose us, testimonials, enquiry call to action
/courses	Courses	Grid with category filter, search, pagination
/courses/:slug	Course Detail	Overview, syllabus accordion, price, Enroll button, enquiry form
/about	About	Story, mission, team
/contact	Contact	Form, address, map, phone
/login, /register, /forgot-password	Auth	Forms with validation

User (login required)

Route	Page
/dashboard	Summary of enrolled courses
/my-courses	List of purchased courses
/learn/:courseId	Lesson list with video player
/payments	Payment history and receipts
/profile	Edit profile, change password

Admin (admin role required)

Route	Page
/admin	Stats cards and recent activity
/admin/courses	Table with add, edit, publish toggle, delete
/admin/courses/new, /admin/courses/:id/edit	Course form with syllabus and lessons builder

Route	Page
/admin/students	Student list with block or unblock
/admin/payments	Payments table, filters, refund action
/admin/enquiries	Enquiry inbox with status updates

11. Security Checklist

- Hash passwords with bcrypt; never log or return them
- Store secrets only in environment variables; commit `.env.example`, never `.env`
- Enable CORS only for the real frontend origins
- Use `helmet` for security headers
- Rate-limit login, register, forgot-password and enquiry endpoints
- Validate and sanitize every request body (Zod or Joi); use `express-mongo-sanitize` against NoSQL injection
- Enforce roles on the server for every protected route
- Verify payment signatures and webhook signatures; compute prices on the server
- Serve everything over HTTPS; cookies are `httpOnly`, `secure`, `sameSite`
- Never expose lesson video URLs in public course responses
- Add a central error handler that hides stack traces in production
- Turn on MongoDB Atlas IP access rules and use a least-privilege database user

12. Environments and Deployment

Environment variables

```
# server
PORT=5000
NODE_ENV=production
MONGO_URI=
JWT_ACCESS_SECRET=
JWT_REFRESH_SECRET=
CLIENT_URL=https://www.yourdomain.com
RAZORPAY_KEY_ID=
RAZORPAY_KEY_SECRET=
RAZORPAY_WEBHOOK_SECRET=
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
EMAIL_USER=
EMAIL_PASS=

# client
VITE_API_URL=https://api.yourdomain.com/api
VITE_RAZORPAY_KEY_ID=
```

Hosting plan (free to start)

Part	Host	Notes
Frontend	Vercel or Netlify	Auto-deploy from GitHub
Backend	Render free web service	Sleeps when idle, so the first request is slow
Database	MongoDB Atlas M0	Free shared cluster
Images	Cloudinary	Free tier
Domain	GoDaddy	Add DNS records: root and www to the frontend host, `api` subdomain to the backend host

GoDaddy shared (cPanel) hosting is mainly for PHP sites and is a poor fit for a Node backend, so use GoDaddy for the domain and DNS, or choose its VPS plan if the client insists on hosting there.

Environments

- **Local:** your machine, Razorpay test keys, a separate Atlas database
- **Production:** live host; switch to Razorpay live keys only after KYC and a full test pass
- Use two git branches: `dev` for work in progress and `main` for what is live

13. Scalability Plan

Stage	Action
Launch	Free tiers as above
First paying students	Move the backend to a paid always-on instance to remove cold starts
Growth	Upgrade Atlas to M10, add caching (Redis) for course lists, serve images through a CDN
Large traffic	Run multiple backend instances behind a load balancer (works because the API is stateless), add database read replicas

Design habits that keep this possible: stateless JWT auth, pagination everywhere, proper indexes, images and video stored off-server, all config in environment variables, and a clear route, controller, service layering.

14. Development Roadmap

Phase	Work	Duration
0. Discovery	Confirm scope, content, branding and payment account with the client	1 to 2 days
1. Design	Sitemap and Figma wireframes	3 days
2. Backend core	Project setup, models, auth, roles, seed admin	4 days
3. Course module	Course CRUD, public listing, search and filters, Cloudinary upload	3 days
4. Payments	Razorpay order, verify, webhook, enrollment, receipts	4 days
5. Frontend public	Home, Courses, Detail, About, Contact, Auth pages	6 days

Phase	Work	Duration
6. User dashboard	My courses, course player, payment history, profile	3 days
7. Admin dashboard	Stats, courses, students, payments, enquiries	4 days
8. Testing and polish	Payment test cases, security pass, responsive fixes, SEO tags	3 days
9. Deploy and handover	Production setup, domain DNS, live payment keys, walkthrough	2 days

Roughly 5 to 6 weeks part-time. Build in this order so something demoable exists early: auth, then courses, then payments, then dashboards.

15. Items to Confirm with the Client

1. Is Razorpay acceptable, and does the business have the KYC documents ready?
2. Are courses live-class based, recorded video, or both? (This decides how lessons are delivered.)
3. Are refunds allowed, and under what policy?
4. Who supplies course content, logo and brand colors?
5. Does the client already own the domain, and who pays for it and for any paid hosting later?
6. Are GST invoices required on payments?
7. Payment terms for the freelance work: advance, milestones and final handover.

16. Quick Start Commands

```
# backend
mkdir server && cd server && npm init -y
npm i express mongoose dotenv cors helmet cookie-parser bcryptjs jsonwebtoken
zod express-rate-limit express-mongo-sanitize razorpay cloudinary multer
nodemailer
npm i -D nodemon

# frontend
npm create vite@latest client -- --template react
cd client && npm i react-router-dom axios @tanstack/react-query react-hook-
form zod tailwindcss
```